Table of Contents

1. Introduction

Abusive email content contributes to 87.1–90.2% of the total email content. This has resulted in security risks, increased requirement of storage, and has violated the privacy of the recipients. (Jain, G., et al. 2019).

The focus on this project is the analysis of spam emails and developing a classification model to distinguish between legitimate (ham) and spam emails. The analysis involves identifying patterns and characteristics within spam messages, text processing and vectorizing, and implementing three different spam filtering models: Logistic Regression, Support Vector Machines (SVM), and Long Short-Term Memory (LSTM) to identify the most suitable and viable solution.

For the purpose of making precise predictions, the models are trained on email data that has been transformed into numerical vectors of a certain length, thereby allowing for a wider examination of the text features involved in spam classification.

By comparing these models, this report aims to determine the most effective spam detection method. useful insights for the improvement of spam filtering systems, ensuring enhanced email security, and reducing the risks associated with malicious emails.

1.1 Problem Statement

This report has the goal of assessing spam email filtering models in an attempt to identify the most effective solution. The key objectives include:

- Identifying commonalities between spam emails
- Implementing different classification models.
- Evaluation of each model using performance metrics.
- Proposing the most effective spam detection approach.

2. Reasonable Solutions

2.1 Solution 1 – Support Vector Machine (SVM)

2.1.1 Description

Support Vector Machine (SVM) is a supervised learning algorithm commonly used for text classification such as spam detection. In this report, a linear kernel was used due to its efficiency in handling high-dimensional text data. This method is less ideal for large data sets, but effective for smaller data sets while not being computationally demanding.

2.1.2 Examples of Prediction

```
[32] pred = predict_labels(clfs['SVC'], x_test)
     conf_mat = confusion_matrix(y_test, pred)
     print(conf_mat)

     sns.heatmap(conf_mat, square = True, annot = True, robust = True)
     plt.show()
```

```
[[1438    2]
 [  32  200]]
```

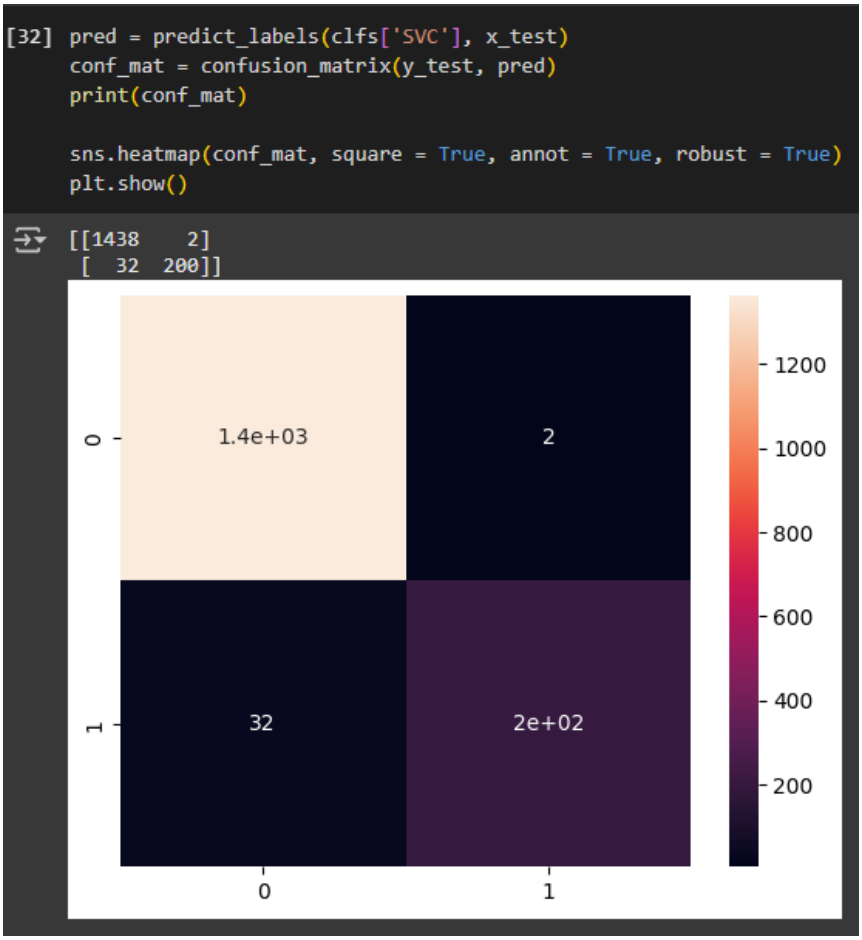

Figure 2.1.1 SVC Method Code

2.2 Solution 2 – Long Short-Term Memory (LSTM)

2.2.1 Description

Long-Short-Term Memory (LSTM) is a variant of the Recursive Neural Network (RNN) with the purpose of handling sequential data and displaying a proficiency in language processing, inclusive of spam filtering. LSTM has the capabilities of removing and adding information to the cell state, memorising more information with longer sequences, and utilises non-linear activation functions, distinguishing it from RNN

Dependencies between words and phrases within emails are captured by LSTM networks sequentially. Memory cells comprising of three gates; input, forget and output each have unique algorithms (Figure 2.2.1) enabling information retainment while discarding irrelevant data. Input within an email must be embedded for this method to read the information by using tools such as Word2Vec and GloVe prior to LSTM's deployment.

INPUT Text Documents, Max Length = 160
OUTPUT class label Spam or Ham
(1) **For** each (Document) input x at time t do

Input gate $i_t = \sigma(W_i x_i + U_i s_{t-1} + b_i)$

Candidate $\bar{C} = \tanh(W_c x_t + U_c s_{t-1} + b_c)$

Forget gate $f_t = (W_f x_t + U_f s_f + b_f)$ d

The Output gate $o_t = \sigma(W_o x_i + U_o s_{t-1} + b_o)$

$s_t = o_t \cdot tanh C_t$

End For

(2) Apply Softmax function on s_t to get the output label

Figure 2.2.1: LSTM Algorithm (Jain, G., et al. 2019.).

2.2.2 Examples of Prediction

LTSM presents a strong line of defence against unwanted emails derived from its precision and consistency in spam identification. Multi-layered training enables deep learning models such as LTSM to learn and functionality. Figure 2.2.2 displays the improvement in performance by incorporating multiple layers of training for the model to learn from.

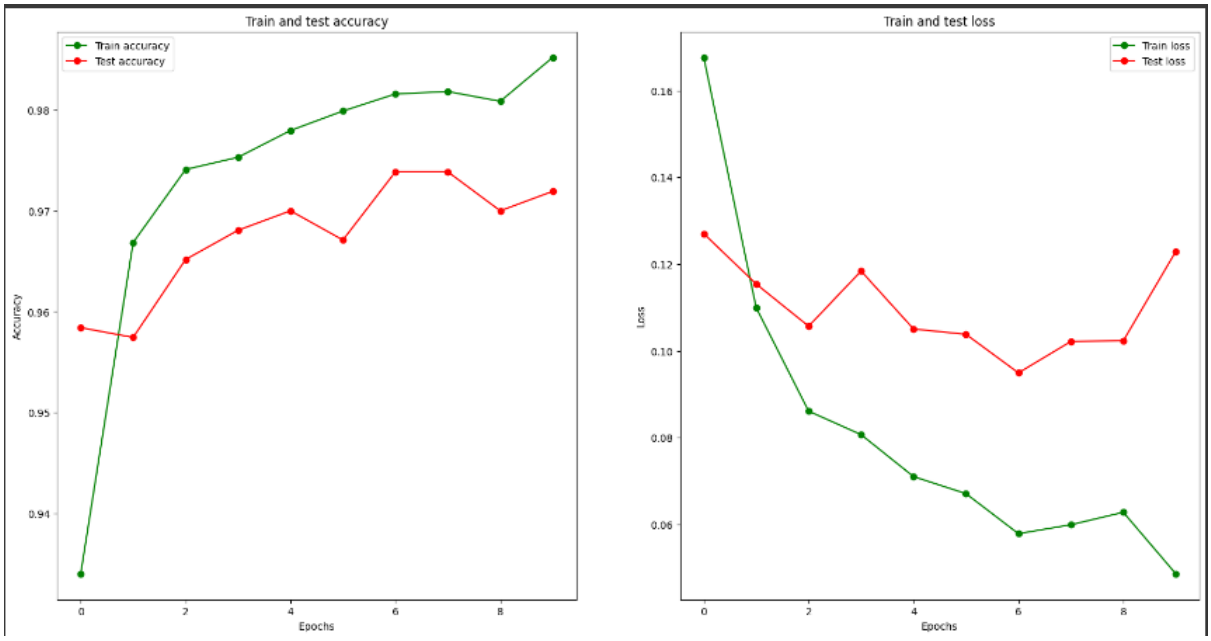

Figure (2.2.2) Accuracy increases with multiple layers

2.2.3 Part of key codes to show the method.

```
model = Sequential()

model.add(Embedding(6013, output_dim = 100, weights = [embedding_matrix], trainable = False))

model.add(LSTM(units = 128, return_sequences = True, recurrent_dropout = 0.3, dropout = 0.5))

model.add(LSTM(units = 64, recurrent_dropout = 0.3, dropout = 0.5))

model.add(Dense(units = 32, activation = "relu"))

model.add(Dense(1, activation = "sigmoid"))

model.compile(optimizer = tf.keras.optimizers.Adam(learning_rate=0.01), loss = "binary_crossentropy", metrics = ["accuracy"])
```

Figure (2.2.3) Building the deep learning model

```
history = model.fit(train_x, train_y, batch_size=64, validation_data=(test_x, test_y), epochs=10, callbacks = [lr_reduce])

Epoch 1/10
65/65 ━━━━━━━━━━━━━━━━━━━━ 52s 633ms/step - accuracy: 0.8694 - loss: 0.2577 - val_accuracy: 0.9478 - val_loss: 0.1402 - learning_rate: 0.0100
Epoch 2/10
65/65 ━━━━━━━━━━━━━━━━━━━━ 86s 700ms/step - accuracy: 0.9693 - loss: 0.0996 - val_accuracy: 0.9603 - val_loss: 0.1196 - learning_rate: 0.0100
Epoch 3/10
65/65 ━━━━━━━━━━━━━━━━━━━━ 34s 522ms/step - accuracy: 0.9743 - loss: 0.0852 - val_accuracy: 0.9632 - val_loss: 0.1034 - learning_rate: 0.0100
Epoch 4/10
65/65 ━━━━━━━━━━━━━━━━━━━━ 40s 612ms/step - accuracy: 0.9750 - loss: 0.0742 - val_accuracy: 0.9729 - val_loss: 0.0982 - learning_rate: 0.0100
Epoch 5/10
65/65 ━━━━━━━━━━━━━━━━━━━━ 39s 587ms/step - accuracy: 0.9821 - loss: 0.0576 - val_accuracy: 0.9662 - val_loss: 0.1060 - learning_rate: 0.0100
Epoch 6/10
65/65 ━━━━━━━━━━━━━━━━━━━━ 42s 604ms/step - accuracy: 0.9780 - loss: 0.0677 - val_accuracy: 0.9739 - val_loss: 0.1017 - learning_rate: 0.0100
Epoch 7/10
65/65 ━━━━━━━━━━━━━━━━━━━━ 43s 630ms/step - accuracy: 0.9832 - loss: 0.0529 - val_accuracy: 0.9681 - val_loss: 0.1212 - learning_rate: 0.0100
Epoch 8/10
 4/65 ━━━━━━━━━━━━━━━━━━━━ 26s 432ms/step - accuracy: 0.9896 - loss: 0.0240
```

Figure (2.2.4) Training the model with multiple layers

```
prediction = model.predict(test_x)
classes_pred = (prediction >= 0.5).astype(int)

print(classification_report(test_y, classes_pred))
```

```
33/33 ━━━━━━━━━━━━━━━━━━━━ 3s 65ms/step
              precision    recall  f1-score   support

           0       0.97      1.00      0.98       896
           1       0.97      0.83      0.89       138

    accuracy                           0.97      1034
   macro avg       0.97      0.91      0.94      1034
weighted avg       0.97      0.97      0.97      1034
```

Figure 2.2.5: Model's precision

2.3 Solution 3 – Logistic Regression (Basic Machine Learning)

2.3.1 Description

Logistic Regression(LR) falls under the category of a supervised machine learning algorithm. The function of this model is to compare data factors and categorise them into classes. LR is extremely efficient in the calculation of probabilities

2.3.2 Examples of Prediction

Figure 2.3.1 displays the data from testing logistical regression.

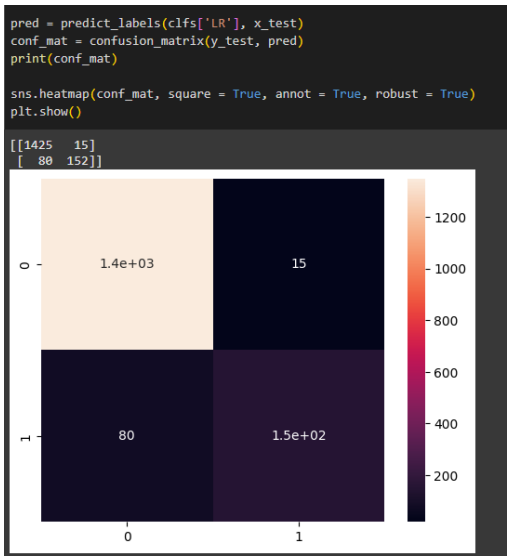

Figure 2.3.1 Logistic Regression data.

3. Metrics & Results

To investigate the effectiveness of LSTM, SVM and Logistic Regression, test cases were used to evaluate four metrics. These metrics provide insights into how well each model differentiates between spam and non-spam (ham) emails.

- **Accuracy** measures the overall proportion of correctly classified emails, indicating the percentage of correctly classified emails.
- **Precision** represents the proportion of correctly identified spam emails out of all emails predicted as spam. A high precision score suggests that the model effectively minimizes false positives.
- **Recall** indicates the model's ability to identify all actual spam emails. A higher recall means fewer spam emails are misclassified as ham.
- **F1-score** draws a comparison between precision and recall, providing a balanced measure of both metrics.

3.1 Model Performance Scores

The following table contains performance scores from each of the three models.

Model	Accuracy	Precision	Recall	F1-score
SVM	97.97%	99.09%	80.26%	92.16%
Logistic Regression (LR)	94.31%	91.07%	65.51%	76.10%
Long Short Term Memory (LSTM)	96.35%	100%	73.76%	84.86%

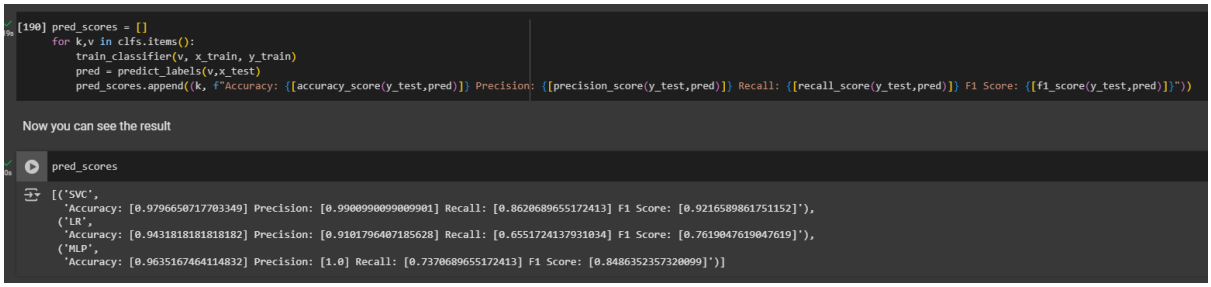

Figure 3.1: Prediction Results

3.2 Results

SVM excelled in accuracy, with a score of 97.97%, which means it classified more emails into the right class. SVM performed strong in precision (99.09%) and f1 score (92.16%), however its recall value (80.26%) is lower than expected, demonstrating that some spam messages were misclassified as ham.

Logistic Regression fell short on performance across all metrics with a recall rate of 65.51%, indicating that it missed a notable proportion of true spam emails. LR's precision rate was respectable at 91.07%, which suggests that whenever the model predicted an email as spam, it was mostly correct.

LSTM performed great overall with an accuracy of 96.35%. A perfect precision score (100%) indicates that it made no false positive mistakes, although its recall score (73.76%) says that it missed some spam emails. LSTM's F1-score of 84.86%, an excellent trade-off between precision and recall.

A model with high precision but low recall (such as LTSM) is good at filtering out false alarms but may let through many spam emails. On the contrary, a model with high recall but compromised precision (such as SVM) catches more spam but may misclassify legitimate emails. With these trade-offs in consideration, SVM is suited for spam filtering for a corporate setting as it achieves a good balance between precision and recall.

The Matplot graph displayed in Figure 3.2 provides a visualisation of each of the model's scores and allows for easy comparison between them.

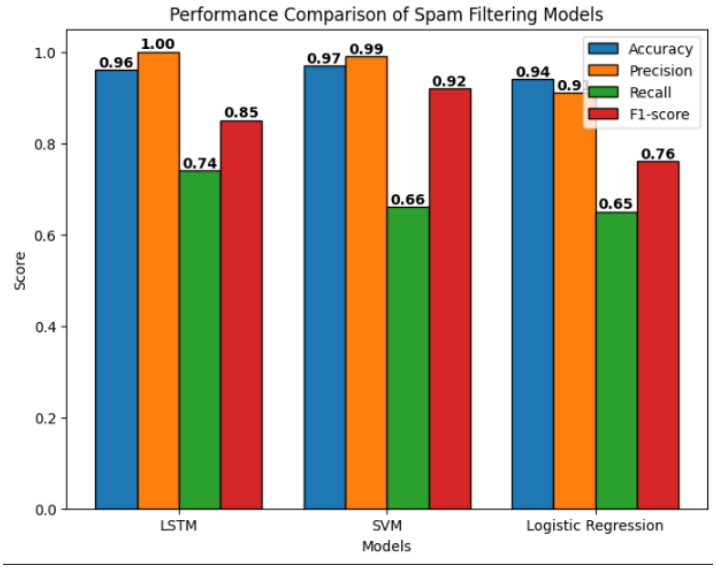

Figure 3.2: Comparison Between Model Results

```python
import numpy as np
import matplotlib.pyplot as plt

models = ["LSTM", "SVM", "Logistic Regression"]
accuracy = [0.96, 0.97, 0.94]
precision = [1.00, 0.99, 0.91]
recall = [0.74, 0.66, 0.65]
f1_score = [0.85, 0.92, 0.76]

x = np.arange(len(models))
width = 0.2

fig, ax = plt.subplots(figsize=(8, 6))
rects1 = ax.bar(x - 1.5 * width, accuracy, width, label="Accuracy")
rects2 = ax.bar(x - 0.5 * width, precision, width, label="Precision")
rects3 = ax.bar(x + 0.5 * width, recall, width, label="Recall")
rects4 = ax.bar(x + 1.5 * width, f1_score, width, label="F1-score")
s
def add_labels(rects):
    for rect in rects:
        height = rect.get_height()
        ax.text(
            rect.get_x() + rect.get_width() / 2,
            height,
            f'{height:.2f}',
            ha='center',
            va='bottom',
            fontsize=10,
            fontweight='bold'
        )

add_labels(rects1)
add_labels(rects2)
add_labels(rects3)
add_labels(rects4)

ax.set_xlabel("Models")
ax.set_ylabel("Score")
ax.set_title("Performance Comparison of Spam Filtering Models")
ax.set_xticks(x)
ax.set_xticklabels(models)
ax.legend()

plt.show()
```

Figure 3.3: Code for Bar Graph

```
[('SVC', [0.9796650717703349]),
 array([[1438,    2],
        [  32,  200]]),
 ('LR', [0.9431818181818182]),
 array([[1425,   15],
        [  80,  152]]),
 ('MLP', [0.9635167464114832]),
 array([[1440,    0],
        [  61,  171]])]
```

Figure 3.4: Confusion Matrix

```
[327] svc = SVC(kernel='linear', gamma=1.0)
      lrc = LogisticRegression(solver='liblinear', penalty='l1')
      mlp = MLPClassifier(solver='adam', alpha=1e-5, hidden_layer_sizes=(5, 2), random_state=1)

[328] clfs = {'SVC' : svc,'LR': lrc, 'MLP': mlp}
```

Figure (3.5): Classifiers and Parameters Used

4. Reflection

This project provided valuable insights into spam detection techniques and their practical applications in cybersecurity. An important takeaway was learning about the compromises between classical machine learning algorithms and deep learning methods. Logistic Regression and SVM are efficient and effective computationally for simpler classification problems but lack the capability to model contextual dependencies in text effectively. On the contrary, LSTM provided remarkable enhancement in spam detection, due to its capacity to learn.

Obstacles encountered such as imbalanced data, fine-tuning for correct results, and optimizing code needed addressing. These challenges involved experimenting with different classifiers, code, and so on.

Ethical Considerations

Ethical considerations are a fundamental component of designing and implementing anti-spam technology and must be taken into account.

Non-Discrimination

Spam filtering techniques must be implemented with fairness in-mind and not directly impact any one person or group negatively. (Vijayakumar, B., Thomas, C. 2024).

Respect for Privacy

Individual privacy rights for users must adhered to and protected and it is the service provider's responsibility to provide a secure environment.

False Positives

Incorrectly labelling legitimate emails as spam can disrupt communication and lead to delays in critical responses.

5. Conclusion

Amongst the three models that were tested, LSTM performed the most effective for spam classification and is the recommended solution for spam-email filtering. Its ability to learn from sequential patterns and read whole sentences, rather than segments, puts it far ahead of traditional models like Logistic Regression and SVM. However, because of computational constraints and deployment considerations, SVM can also serve as a sufficient alternative.

References

Jain, G., Sharma, M. & Agarwal, B. Optimizing semantic LSTM for spam detection. *Int. j. inf. tecnol.* **11**, 239–250 (2019). https://doi.org/10.1007/s41870-018-0157-5

John-Africa , E., & Emmah, V. T. (2022). Performance Evaluation of LSTM and RNN Models in the Detection of Email Spam Messages. *European Journal of Information Technologies and Computer Science*, *2*(6), 24–30. https://doi.org/10.24018/compute.2022.2.6.80

P. Bhatti, Z. Jalil and A. Majeed, "Email Classification using LSTM: A Deep Learning Technique," 2021 International Conference on Cyber Warfare and Security (ICCWS), Islamabad, Pakistan, 2021, pp. 100-105, doi: 10.1109/ICCWS53234.2021.9703084.

Saumya, S., Singh, J.P. Spam review detection using LSTM autoencoder: an unsupervised approach. *Electron Commer Res* **22**, 113–133 (2022). https://doi.org/10.1007/s10660-020-09413-4

Vijayakumar, B., Thomas, C. (2024). The ethics of envisioning spam free email inboxes. *AI Ethics*. https://doi.org/10.1007/s43681-024-00526-2